Java Advanced Interview Guide

Spring Security & JWT • Kafka • SQL & Database • REST API

Complete reference for 4+ years experience interviews

1. Spring Security & JWT

Q: What is Spring Security?

Spring Security is a powerful and customizable authentication and authorization framework for Java applications. It is the standard for securing Spring-based applications.

Key features:

• Authentication — Who are you? (login)

• Authorization — What can you do? (roles/permissions)

• Protection against CSRF, session fixation, clickjacking

• Added via spring-boot-starter-security dependency

Q: What is the Spring Security Filter Chain?

The Filter Chain is a series of filters that every HTTP request passes through before reaching your controller. Each filter handles a specific security concern.

Key filters in order:

1. **SecurityContextPersistenceFilter** — loads SecurityContext from session
2. **UsernamePasswordAuthenticationFilter** — handles login form submissions
3. **JwtAuthenticationFilter** — (custom) validates JWT token
4. **ExceptionTranslationFilter** — handles AuthenticationException and AccessDeniedException
5. **FilterSecurityInterceptor** — checks authorization for the request

```
Request → Filter1 → Filter2 → Filter3 → ... → Controller ↓ (if any filter rejects) 401 Unauthorized / 403 Forbidden
```

Q: What is JWT (JSON Web Token)?

JWT is a compact, self-contained token used for securely transmitting information between parties as a JSON object. It is digitally signed so it can be verified and trusted.

Structure — 3 parts separated by dots:

• **Header:** algorithm and token type

• **Payload:** claims (sub, iat, exp, role etc.)

• **Signature:** HMACSHA256(base64(header) + base64(payload) + secret)

Example: xxxxx.yyyyy.zzzzz

```
// JWT Payload example { "sub": "john@gmail.com", "iat": 1716000000, "exp": 1716086400, "role": "ROLE_USER" }
```

Q: How does JWT Authentication flow work in Spring Boot?

1. User sends login request with username and password.
2. Server validates credentials and generates a JWT token.
3. Server returns JWT token to the client.
4. Client stores token (localStorage or cookie).
5. Client sends JWT in every request header: Authorization: Bearer <token>
6. Server's JwtAuthenticationFilter intercepts the request, extracts and validates the token.
7. If valid, sets Authentication in SecurityContext and allows request to proceed.
8. Controller handles the request.

```
Client Server |--- POST /login {username, password} ---> |<-- 200 OK {token: "eyJ..."} ----------- | |--- GET /api/users | Authorization: Bearer eyJ... -------> | (JwtFilter validates token) |<-- 200 OK {users: [...]} -------------
```

Q: How to implement JWT in Spring Boot? (Step by step)

Step 1: Add dependencies in pom.xml

Step 2: Create JwtUtil class (generate + validate token)

Step 3: Create JwtAuthenticationFilter (extends OncePerRequestFilter)

Step 4: Create SecurityConfig (configure filter chain)

Step 5: Create AuthController (login endpoint)

```
// Step 1 - pom.xml <dependency> <groupId>io.jsonwebtoken</groupId> <artifactId>jjwt-api</artifactId> <version>0.11.5</version> </dependency>
```

```
// Step 2 - JwtUtil.java @Component public class JwtUtil { private String SECRET = "mySecretKey123"; private long EXPIRY = 86400000; // 24 hours public String generateToken(String username) { return Jwts.builder() .setSubject(username) .setIssuedAt(new Date()) .setExpiration(new Date(System.currentTimeMillis() + EXPIRY)) .signWith(SignatureAlgorithm.HS256, SECRET) .compact(); } public String extractUsername(String token) { return Jwts.parser().setSigningKey(SECRET) .parseClaimsJws(token).getBody().getSubject(); } public boolean validateToken(String token, UserDetails user) { String username = extractUsername(token); return username.equals(user.getUsername()) && !isTokenExpired(token); } private boolean isTokenExpired(String token) { Date expiry = Jwts.parser().setSigningKey(SECRET)
```

```
.parseClaimsJws(token).getBody().getExpiration(); return expiry.before(new Date()); } }
```

```
// Step 3 - JwtAuthenticationFilter.java @Component public class JwtAuthenticationFilter extends OncePerRequestFilter { @Autowired private JwtUtil jwtUtil; @Autowired private UserDetailsService userDetailsService; @Override protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain) throws ServletException, IOException { String authHeader = request.getHeader("Authorization"); if (authHeader != null && authHeader.startsWith("Bearer ")) { String token = authHeader.substring(7); String username = jwtUtil.extractUsername(token); if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) { UserDetails userDetails = userDetailsService.loadUserByUsername(username); if (jwtUtil.validateToken(token, userDetails)) { UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken( userDetails, null, userDetails.getAuthorities()); SecurityContextHolder.getContext() .setAuthentication(authToken); } } } chain.doFilter(request, response); } }
```

```
// Step 4 - SecurityConfig.java @Configuration @EnableWebSecurity public class SecurityConfig { @Autowired private JwtAuthenticationFilter jwtFilter; @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http .csrf().disable() .authorizeHttpRequests() .requestMatchers("/api/auth/**").permitAll() .anyRequest().authenticated() .and() .sessionManagement() .sessionCreationPolicy(SessionCreationPolicy.STATELESS) .and() .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class); return http.build(); } @Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); } }
```

```
// Step 5 - AuthController.java @RestController @RequestMapping("/api/auth") public class AuthController { @Autowired private AuthenticationManager authManager; @Autowired private JwtUtil jwtUtil; @PostMapping("/login") public ResponseEntity<?> login(@RequestBody LoginRequest request) { authManager.authenticate( new UsernamePasswordAuthenticationToken( request.getUsername(), request.getPassword())); String token = jwtUtil.generateToken(request.getUsername()); return ResponseEntity.ok(new LoginResponse(token)); } }
```

Q: What is the difference between Authentication and Authorization?

Authentication: Verifying who the user is. (Login — username/password check)

Authorization: Verifying what the user can do. (Role-based access — ADMIN, USER)

Authentication happens first, then Authorization.

```
// Authorization - role based @PreAuthorize("hasRole('ADMIN')") @GetMapping("/admin/users") public List<User> getAllUsers() { ... } @PreAuthorize("hasAnyRole('ADMIN', 'USER')") @GetMapping("/profile") public User getProfile() { ... }
```

Q: What is CSRF and how does Spring Security handle it?

CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks a user's browser into making unwanted requests to another site where the user is authenticated.

Spring Security enables CSRF protection by default for web apps. For stateless REST APIs using JWT, CSRF is disabled because JWT tokens are not automatically sent by browsers (unlike cookies).

```
// Disable CSRF for REST APIs with JWT http.csrf().disable(); // Enable CSRF for web apps (default) // Spring adds CSRF token to forms automatically
```

Q: What is BCrypt and why is it used for passwords?

BCrypt is a password hashing function. It is used to store passwords securely because:

• It is a one-way hash — you cannot reverse it to get the original password.
• It includes a salt automatically — prevents rainbow table attacks.
• It is slow by design — makes brute-force attacks difficult.
• Cost factor can be increased over time as hardware gets faster.

```
@Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); // default cost = 10 } // Encoding String hashed = passwordEncoder.encode("myPassword"); // Matching (during login) boolean match = passwordEncoder.matches("myPassword", hashed); // true
```

---

2. Apache Kafka

Q: What is Apache Kafka?

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications. It is highly scalable, fault-tolerant, and can handle millions of messages per second.

Common use cases:

• Microservices async communication
• Real-time event processing
• Log aggregation
• Order processing systems (e.g., Zomato, Swiggy)

Q: What are the core components of Kafka?

Producer: Sends (publishes) messages to a Kafka topic.
Consumer: Reads (subscribes) messages from a Kafka topic.
Topic: A category/channel where messages are stored. Like a table in a database.
Partition: Topics are split into partitions for parallelism and scalability.
Broker: A Kafka server that stores and serves messages.
Consumer Group: A group of consumers that share the work of reading from a topic. Each partition is consumed by only one consumer in a group.
Offset: A unique ID for each message within a partition. Consumers track their position using

offset.

ZooKeeper/KRaft: Manages broker metadata and leader election (ZooKeeper being replaced by KRaft in newer versions).

```
Producer --> [Topic: orders] ■■■ Partition 0: msg1, msg2, msg3 ■■■ Partition 1: msg4, msg5, msg6 ■■■ Partition 2: msg7, msg8, msg9 | Consumer Group: order-service ■■■ Consumer 1 reads Partition 0 ■■■ Consumer 2 reads Partition 1 ■■■ Consumer 3 reads Partition 2
```

Q: What is the difference between Kafka and RabbitMQ?

Kafka:

• Log-based message store — messages are retained even after consumption.

• Pull-based — consumers pull messages at their own pace.

• High throughput — millions of messages/second.

• Best for: event streaming, audit logs, replaying events.

RabbitMQ:

• Queue-based — messages deleted after consumption.

• Push-based — broker pushes messages to consumers.

• Better for: task queues, request-reply patterns, complex routing.

• Best for: job queues, notifications, simple async tasks.

Q: How to implement Kafka Producer in Spring Boot?

```
// pom.xml <dependency> <groupId>org.springframework.kafka</groupId> <artifactId>spring-kafka</artifactId> </dependency>
```

```
# application.properties spring.kafka.bootstrap-servers=localhost:9092 spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.StringSerializer
```

```
// Producer Service @Service public class KafkaProducerService { @Autowired private KafkaTemplate<String, String> kafkaTemplate; public void sendMessage(String topic, String message) { kafkaTemplate.send(topic, message); System.out.println("Message sent: " + message); } } // Controller @RestController public class OrderController { @Autowired private KafkaProducerService producer; @PostMapping("/order") public String placeOrder(@RequestBody String order) { producer.sendMessage("order-topic", order); return "Order placed!"; } }
```

Q: How to implement Kafka Consumer in Spring Boot?

```
# application.properties spring.kafka.bootstrap-servers=localhost:9092 spring.kafka.consumer.group-id=order-service spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer spring.kafka.consumer.value-deserializer=org.apache.kafka.common.serialization.StringDeserializer spring.kafka.consumer.auto-offset-reset=earliest
```

```
// Consumer Service @Service public class KafkaConsumerService { @KafkaListener(topics = "order-topic", groupId = "order-service") public void consumeMessage(String message) { System.out.println("Message received: " +
```

```
message); // process order... } // Listen to multiple topics
@KafkaListener(topics = {"order-topic", "payment-topic"}, groupId =
"notification-service") public void consumeMultiple(String message) {
System.out.println("Notification: " + message); } }
```

Q: How to send and consume JSON objects with Kafka?

```
# application.properties spring.kafka.producer.value-serializer=org.springframe
work.kafka.support.serializer.JsonSerializer spring.kafka.consumer.value-deseri
alizer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.trusted.packages=*
```

```
// Order model public class Order { private String orderId; private String
product; private double amount; // getters, setters, constructors } // Producer
@Autowired private KafkaTemplate<String, Order> kafkaTemplate; public void
sendOrder(Order order) { kafkaTemplate.send("order-topic", order); } // Consumer
@KafkaListener(topics = "order-topic", groupId = "order-service") public void
consumeOrder(Order order) { System.out.println("Order received: " +
order.getOrderId()); }
```

Q: What is a Consumer Group in Kafka?

A Consumer Group is a group of consumers that work together to consume messages from a topic.

Rules:

• Each partition is assigned to only ONE consumer within a group.

• If consumers > partitions — some consumers will be idle.

• If consumers < partitions — some consumers handle multiple partitions.

• Different consumer groups each get ALL messages (independent reading).

Real example: OrderService and NotificationService both consume from order-topic — each gets every message independently.

Q: What is Offset in Kafka?

Offset is a unique sequential ID assigned to each message within a partition. Consumers use offsets to track which messages they have already processed.

auto-offset-reset options:

• earliest — start reading from the beginning of the topic.

• latest — start reading only new messages (default).

Commit strategies:

• Auto commit — Kafka auto-commits offset periodically.

• Manual commit — Consumer explicitly commits offset after processing.

```
// Manual commit example @KafkaListener(topics = "order-topic", groupId =
"order-service") public void consume(String message, Acknowledgment ack) { try {
processMessage(message); ack.acknowledge(); // commit only after successful
```

```
processing } catch (Exception e) { // don't ack - message will be redelivered }
}
```

Q: What is Kafka Partition and why is it important?

A partition is a subdivision of a topic that enables parallelism.

• Each partition is an ordered, immutable log of messages.
• More partitions = more parallelism = higher throughput.
• Messages with the same key always go to the same partition (key-based routing).
• Replication factor determines how many copies of each partition exist across brokers for fault tolerance.

```
// Sending to specific partition with key kafkaTemplate.send("order-topic",
"user123", orderJson); // All orders for "user123" go to same partition //
Ensures ordering for that user
```

Q: What happens if a Kafka Consumer fails?

• If a consumer fails, Kafka rebalances — reassigns that consumer's partitions to other consumers in the group.
• If auto-commit was used, unprocessed messages may be lost or reprocessed.
• With manual commit, the offset is not committed — messages are reprocessed after recovery.
• Dead Letter Topic (DLT) — failed messages after max retries are sent to a separate DLT for investigation.

```
// Dead Letter Topic config @Bean public DefaultErrorHandler errorHandler(
DeadLetterPublishingRecoverer recoverer) { return new
DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 3)); // retry 3 times, 1s
apart } @Bean public DeadLetterPublishingRecoverer recoverer(
KafkaTemplate<Object, Object> template) { return new
DeadLetterPublishingRecoverer(template); // sends to topic: order-topic.DLT }
```

---

3. SQL & Database

Q: What are the types of SQL Joins?

INNER JOIN: Returns rows that have matching values in both tables.
LEFT JOIN (LEFT OUTER): Returns all rows from the left table + matched rows from right. NULL if no match on right.
RIGHT JOIN (RIGHT OUTER): Returns all rows from the right table + matched rows from left. NULL if no match on left.
FULL OUTER JOIN: Returns all rows from both tables. NULL where no match.
CROSS JOIN: Returns cartesian product — every row from left with every row from right.
SELF JOIN: A table joined with itself.

```
-- INNER JOIN SELECT e.name, d.dept_name FROM employees e INNER JOIN departments
d ON e.dept_id = d.id; -- LEFT JOIN (all employees, even without department)
```

```
SELECT e.name, d.dept_name FROM employees e LEFT JOIN departments d ON e.dept_id = d.id; -- SELF JOIN (employee and their manager) SELECT e.name AS employee, m.name AS manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.id;
```

Q: What is the difference between WHERE and HAVING?

WHERE: Filters rows BEFORE grouping. Cannot use aggregate functions.

HAVING: Filters groups AFTER GROUP BY. Can use aggregate functions (COUNT, SUM, AVG).

```
-- WHERE - filter before grouping SELECT dept_id, COUNT(*) as emp_count FROM employees WHERE salary > 50000 -- filter rows first GROUP BY dept_id; -- HAVING - filter after grouping SELECT dept_id, COUNT(*) as emp_count FROM employees GROUP BY dept_id HAVING COUNT(*) > 5; -- filter groups
```

Q: What is Indexing in SQL? Types?

An index is a data structure that speeds up data retrieval. It works like a book's index — instead of scanning all rows, the DB uses the index to find data faster.

Types:

- **Primary Index:** Automatically created on Primary Key. Unique and not null.
- **Unique Index:** Ensures all values in the column are unique.
- **Composite Index:** Index on multiple columns.
- **Full-Text Index:** For searching text content.

When NOT to use index:

- Small tables — full scan is faster.
- Columns with low cardinality (e.g., gender: M/F).
- Tables with heavy INSERT/UPDATE — indexes slow down writes.

```
-- Create index CREATE INDEX idx_emp_name ON employees(name); -- Composite index CREATE INDEX idx_dept_salary ON employees(dept_id, salary); -- Unique index CREATE UNIQUE INDEX idx_emp_email ON employees(email); -- Drop index DROP INDEX idx_emp_name ON employees; -- Check if index is used (EXPLAIN) EXPLAIN SELECT * FROM employees WHERE name = 'John';
```

Q: What is Query Optimization in SQL?

Query optimization is the process of improving SQL query performance.

Key techniques:

- Use indexes on frequently searched columns.
- Avoid SELECT * — select only needed columns.
- Use WHERE clause to filter early and reduce rows.
- Avoid functions on indexed columns in WHERE (disables index use).
- Use JOIN instead of subqueries where possible.
- Use LIMIT to restrict result set.
- Use EXPLAIN to analyze query execution plan.

• Avoid N+1 queries — use JOIN or batch fetch.
• Use connection pooling (HikariCP in Spring Boot).

```
-- Bad - function on indexed column (index not used) SELECT * FROM employees
WHERE YEAR(hire_date) = 2023; -- Good - range on indexed column (index used)
SELECT * FROM employees WHERE hire_date BETWEEN '2023-01-01' AND '2023-12-31';
-- Bad - SELECT * SELECT * FROM employees; -- Good - specific columns SELECT id,
name, salary FROM employees; -- EXPLAIN to see query plan EXPLAIN SELECT * FROM
employees WHERE dept_id = 5;
```

Q: What are Transactions and ACID properties?

Transaction: A sequence of operations treated as a single unit — either ALL succeed or ALL fail.

ACID Properties:

• **Atomicity:** All operations in a transaction complete or none do (all or nothing).
• **Consistency:** Database moves from one valid state to another valid state.
• **Isolation:** Concurrent transactions don't interfere with each other.
• **Durability:** Committed transactions are permanently saved even after system failure.

```
-- Transaction example BEGIN TRANSACTION; UPDATE accounts SET balance = balance
- 1000 WHERE id = 1; UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
-- If both succeed COMMIT; -- If any fails ROLLBACK;
```

Q: What is the difference between DELETE, TRUNCATE, and DROP?

DELETE: Removes specific rows based on WHERE condition. Can be rolled back. Slower — logs each row deletion. Triggers fire.

TRUNCATE: Removes ALL rows from a table. Cannot be rolled back (in most DBs). Faster — does not log individual rows. Resets auto-increment. Triggers don't fire.

DROP: Removes the entire table including structure, data, indexes, and constraints. Cannot be rolled back.

```
-- DELETE - specific rows DELETE FROM employees WHERE dept_id = 5; -- TRUNCATE -
all rows, keeps structure TRUNCATE TABLE employees; -- DROP - removes entire
table DROP TABLE employees;
```

Q: What are Window Functions in SQL?

Window functions perform calculations across a set of rows related to the current row without collapsing them into groups (unlike GROUP BY). Very common in 4+ year interviews.

```
-- ROW_NUMBER - unique row number per partition SELECT name, dept_id, salary,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rank FROM
employees; -- RANK - same rank for ties, gaps in ranking SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) as rank FROM employees; -- DENSE_RANK - same
rank for ties, no gaps SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary
DESC) as rank FROM employees; -- Find top 1 salary per department SELECT * FROM
( SELECT name, dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY
salary DESC) as rn FROM employees ) t WHERE rn = 1;
```

Q: What is the difference between UNION and UNION ALL?

UNION: Combines results of two queries and removes duplicate rows. Slower due to deduplication.

UNION ALL: Combines results and keeps all rows including duplicates. Faster.

```
-- UNION - removes duplicates SELECT name FROM employees_india UNION SELECT name FROM employees_us; -- UNION ALL - keeps duplicates (faster) SELECT name FROM employees_india UNION ALL SELECT name FROM employees_us;
```

Q: What are Stored Procedures and when to use them?

A stored procedure is a precompiled group of SQL statements stored in the database that can be executed by name.

Benefits: Reusability, better performance (precompiled), reduces network traffic, security (hide logic).

When to use: Complex business logic at DB level, batch operations, repeated operations.

```
-- Create stored procedure DELIMITER // CREATE PROCEDURE GetEmployeesByDept(IN dept_id INT) BEGIN SELECT * FROM employees WHERE department_id = dept_id; END // DELIMITER ; -- Call it CALL GetEmployeesByDept(5); -- In Spring Boot with JPA @Query(value = "CALL GetEmployeesByDept(:deptId)", nativeQuery = true) List<Employee> getByDept(@Param("deptId") int deptId);
```

Q: What are common SQL interview queries?

```
-- 1. Second highest salary SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees); -- 2. Nth highest salary SELECT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET (N-1); -- N=2 gives 2nd highest -- 3. Duplicate records SELECT email, COUNT(*) as count FROM employees GROUP BY email HAVING COUNT(*) > 1; -- 4. Employees with salary > department average SELECT e.name, e.salary, e.dept_id FROM employees e WHERE e.salary > ( SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id ); -- 5. Employees without a manager SELECT * FROM employees WHERE manager_id IS NULL; -- 6. Count employees per department SELECT d.dept_name, COUNT(e.id) as total FROM departments d LEFT JOIN employees e ON d.id = e.dept_id GROUP BY d.dept_name;
```

---

4. REST API Deep Dive

Q: What are HTTP methods and when to use each?

GET: Retrieve data. No body. Safe + Idempotent. Ex: GET /users/1

POST: Create a new resource. Has body. NOT idempotent. Ex: POST /users

PUT: Update/Replace entire resource. Has body. Idempotent. Ex: PUT /users/1

PATCH: Partially update a resource. Has body. Ex: PATCH /users/1

DELETE: Delete a resource. Idempotent. Ex: DELETE /users/1

HEAD: Same as GET but returns only headers, no body.

OPTIONS: Returns supported HTTP methods for a resource.

Q: What is the difference between PUT and PATCH?

PUT: Replaces the ENTIRE resource. If you omit a field, it becomes null/empty.

PATCH: Updates ONLY the fields provided. Other fields remain unchanged.

Both are idempotent — calling them multiple times gives the same result.

```
// PUT - must send entire object PUT /users/1 { "name": "John", "email": "john@gmail.com", // must include all fields "age": 25 } // PATCH - send only changed fields PATCH /users/1 { "email": "newemail@gmail.com" // only update email }
```

Q: What is Idempotency in REST API?

An operation is idempotent if calling it multiple times with the same input produces the same result as calling it once.

Idempotent methods: GET, PUT, DELETE, HEAD, OPTIONS

Non-idempotent: POST (creates new resource each time)

Real-world use: Payment API — if payment request is sent twice due to network retry, idempotency key ensures the user is only charged once.

```
// Idempotency key in header POST /payments Headers: Idempotency-Key: "unique-key-abc123" { "amount": 500, "userId": "user1" } // Same request with same key = same result, no duplicate charge
```

Q: What are HTTP Status Codes? (Must know for interviews)

2xx — Success:

• 200 OK — request succeeded

• 201 Created — resource created (POST)

• 204 No Content — success but no body (DELETE)

3xx — Redirection:

• 301 Moved Permanently

• 302 Found (temporary redirect)

4xx — Client Errors:

• 400 Bad Request — invalid request syntax

• 401 Unauthorized — not authenticated

• 403 Forbidden — authenticated but no permission

• 404 Not Found — resource doesn't exist

• 405 Method Not Allowed

• 409 Conflict — duplicate resource

• 422 Unprocessable Entity — validation failed

5xx — Server Errors:

• 500 Internal Server Error
• 502 Bad Gateway
• 503 Service Unavailable
• 504 Gateway Timeout

Q: What is the difference between 401 and 403?

401 Unauthorized: The user is NOT authenticated — they haven't logged in or token is invalid/expired. Solution: Login again.

403 Forbidden: The user IS authenticated but does NOT have permission to access this resource. Solution: Need different role/permission.

Q: What are REST API best practices?

• Use nouns not verbs in URLs: /users not /getUsers
• Use plural nouns: /users not /user
• Use HTTP methods correctly: GET to read, POST to create, PUT/PATCH to update, DELETE to delete
• Use proper HTTP status codes
• Version your API: /api/v1/users
• Use query params for filtering: /users?role=admin&page;=1
• Use path params for specific resources: /users/{id}
• Always return consistent JSON response structure
• Secure with JWT/OAuth2
• Use HTTPS always
• Implement pagination for large datasets

```
// Good URL design GET /api/v1/users - get all users GET /api/v1/users/1 - get user by id POST /api/v1/users - create user PUT /api/v1/users/1 - update user DELETE /api/v1/users/1 - delete user GET /api/v1/users/1/orders - get orders of user 1 // Consistent response structure { "status": "success", "data": { ... }, "message": "User created successfully", "timestamp": "2024-01-01T10:00:00" }
```

Q: What is the difference between @PathVariable and @RequestParam?

@PathVariable: Extracts value from the URL path. Used for identifying a specific resource.

@RequestParam: Extracts value from query parameters (?key=value). Used for filtering, sorting, pagination.

```
// @PathVariable - part of the URL path @GetMapping("/users/{id}") public User getUser(@PathVariable Long id) { ... } // Called as: GET /users/1 // @RequestParam - query parameter @GetMapping("/users") public List<User> getUsers( @RequestParam(required = false) String role, @RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "10") int size) { ... } // Called as: GET /users?role=admin&page=0&size=10 // Combined @GetMapping("/users/{id}/orders") public List<Order> getUserOrders( @PathVariable Long id, @RequestParam String status) { ... } // Called as: GET /users/1/orders?status=pending
```

Q: What is @RequestBody and @ResponseBody?

@RequestBody: Binds the HTTP request body (JSON) to a Java object. Used in POST/PUT/PATCH.

@ResponseBody: Tells Spring to serialize the return value to JSON and write it to the response body. Already included in @RestController.

```
@PostMapping("/users") public ResponseEntity<User> createUser(@RequestBody User user) { // user object is automatically deserialized from JSON body User saved = userService.save(user); return ResponseEntity.status(201).body(saved); } // ResponseEntity - full control over response @GetMapping("/users/{id}") public ResponseEntity<User> getUser(@PathVariable Long id) { return userService.findById(id) .map(user -> ResponseEntity.ok(user)) .orElse(ResponseEntity.notFound().build()); }
```

Q: What is CORS and how to handle it in Spring Boot?

CORS (Cross-Origin Resource Sharing) is a browser security feature that blocks requests from a different domain/port than the server.

Example: Frontend on http://localhost:3000 calling backend on http://localhost:8080 — browser blocks this by default.

```
// Option 1 - @CrossOrigin on controller @CrossOrigin(origins = "http://localhost:3000") @RestController public class UserController { ... } // Option 2 - Global CORS config @Configuration public class CorsConfig { @Bean public WebMvcConfigurer corsConfigurer() { return new WebMvcConfigurer() { @Override public void addCorsMappings(CorsRegistry registry) { registry.addMapping("/api/**") .allowedOrigins("http://localhost:3000") .allowedMethods("GET","POST","PUT","DELETE") .allowedHeaders("*"); } }; } }
```

Q: What is Pagination and Sorting in Spring Boot REST API?

```
// Using Spring Data JPA Pageable @GetMapping("/users") public ResponseEntity<Page<User>> getUsers( @RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "10") int size, @RequestParam(defaultValue = "id") String sortBy) { Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy).ascending()); Page<User> users = userRepository.findAll(pageable); return ResponseEntity.ok(users); } // Called as: // GET /users?page=0&size=10&sortBy=name // Response includes: // { content: [...], totalPages: 5, // totalElements: 50, pageNumber: 0 }
```